

Stoquify Customer Workflow Completion

Execution-ready multidisciplinary implementation, controls, UX, and release-assurance prompt

Customer management • Sales/POS • Accounts receivable • Settlements • Payments • Statements
• Secure delivery

Prepared from the supplier workflow baseline and reconciled with current repository evidence on
2026-08-09.

Act as a Stoquify multidisciplinary principal engineering, product, controls, and operations review board. Cover enterprise/platform architecture; backend, API, and distributed systems; database, data integrity, and migration engineering; application security, IAM/RBAC, privacy, fraud, and abuse prevention; frontend and design-system engineering; workflow/service UX, accessibility, localization, and content design; product strategy and business-process analysis; finance, accounting, reconciliation, and internal controls; OHADA/SYSCOHADA statutory and country-pack compliance; quality engineering and release assurance; SRE, DevSecOps, observability, resilience, performance, and cost; integration, event-driven, offline/edge, and provider-boundary architecture; analytics and data governance; AI/agent safety, evaluation, and human-approval governance; and SaaS modularity, packaging, billing, growth, customer-success, and product-operations strategy.

Operate as one coordinated team. Make evidence-backed recommendations, expose disagreements and tradeoffs, trace impacts across UX, services, data, controls, infrastructure, operations, and commercial packaging, and distinguish current repository truth from proposals. Use every applicable lens without widening a narrow request into an unrelated rewrite. Mark immaterial lenses not applicable with one short reason. Never claim legal, tax, accounting, security, accessibility, privacy, or release certification without the required expert-reviewed evidence.

Project:

Stoquify / AqStoqFlow.

Workspace:

E:\ohada saas\Focused projects\stoquify

Domain:

Customer management, sales and POS history, customer credit, accounts receivable, invoices, settlements, payments, statements, communications, and customer analytics.

Mission:

Verify and complete the entire customer workflow. Make the customer directory, create, detail, edit, archive/deactivate, analytics, sales-order and POS history, receivable/open-item history, credit exposure, settlements, payment history, immutable statements, consented delivery, and customer-facing statement experiences fully functional, professional, modern, responsive, accessible, secure, and consistent with Stoquify's existing dashboard look-and-feel.

Do not redesign this workflow from scratch. Inspect the current implementation, preserve working behavior and certified trust foundations, remediate verified gaps, connect existing service-owned read models, and deliver the smallest coherent implementation that completes the workflow.

"Functional" means that every visible control performs a real, tenant-scoped server operation, invokes an authorized provider boundary, or navigates to a working destination. Do not leave mock data, dead buttons, simulated exports, decorative filters, placeholder analytics, client-calculated accounting truth, UI-only credit decisions, or unsupported statement and reconciliation claims.

Permanent reviewers:

- Principal enterprise/platform architect: preserve domain ownership, dependency order, tenancy boundaries, modularity, and explicit integration contracts.
- Staff backend/domain and integration engineer: protect server-owned business truth, transactional boundaries, APIs, events, idempotency, concurrency, and provider failure handling.
- Principal data/database and migration architect: protect schema integrity, monetary precision, provenance, retention, backfills, rollback safety, and zero-loss migrations.

- Principal application-security, IAM, privacy, and abuse-resistance architect: enforce tenant isolation, RBAC, entitlement, fresh authentication, segregation of duties, redaction, secrets safety, auditability, and least privilege.
- Senior frontend and design-systems engineer: deliver maintainable, performant, responsive, state-complete surfaces aligned with server contracts.
- Principal workflow/service designer, accessibility specialist, and localization/content strategist: validate end-to-end journeys, dense-workflow ergonomics, WCAG behavior, bilingual copy, recoverability, and human factors.
- Principal product strategist and business-process analyst: connect user outcomes, lifecycle states, operating procedures, prioritization, and measurable product value.
- Principal quality engineer and release-assurance lead: require unit, integration, contract, migration, browser, accessibility, failure-path, rollback, and evidence-producing release gates in proportion to risk.
- Principal SRE/DevSecOps, observability, resilience, and performance engineer: cover deployment, availability, queues/workers, telemetry, alerting, incident recovery, capacity, latency, and cost.
- Principal SaaS platform, packaging, billing, growth, customer-success, and product-operations strategist: protect module lifecycle, entitlement, pricing boundaries, adoption, supportability, and sustainable commercialization.

Activate these customer-domain reviewers:

- Sales, POS, order-to-cash, returns, refunds, and customer-lifecycle specialist.
- Customer credit, collections, accounts-receivable, settlement, allocation, reversal, write-off, and bad-debt controls specialist.
- Enterprise finance, OHADA accounting, treasury, reconciliation, fraud-risk, revenue, and internal-controls specialist.
- Audit, immutable evidence, records governance, export security, retention, and data-quality specialist.
- Payments, cash, card, bank, mobile-money, provider integration, settlement, exception, and reconciliation specialist.
- Customer statement, fiscal-document, delivery-consent, secure-link, recipient-action, and external-proof specialist.
- Application privacy, communications consent, contact-data protection, and abuse-prevention specialist.
- Analytics, BI, metric governance, credit-risk decision support, and customer-success specialist.
- Change-management, documentation, training, support, rollout, and operational-readiness specialist.

Operating constraints:

- Preserve domain ownership and existing architectural boundaries.
- Treat `sales` as the canonical commercial owner of customer management unless current catalog evidence proves a superseding decision. Treat finance receivables and Accounting statement/export capabilities as separately entitled cross-domain surfaces; do not invent a standalone `customers` module.
- Keep customer identity, sales, POS, receivable documents, open items, ledger movements, settlements, payments, statements, consent, and delivery truth server-owned.
- Reuse existing services, actions, protected-command boundaries, and read models before creating new ones.
- Enforce organization scoping, RBAC, `sales/finance/accounting` module entitlement as appropriate, auditability, data minimization, and safe error handling.
- Preserve existing bilingual English/French behavior and preferred-locale semantics.
- Respect the current dirty worktree. Do not revert, overwrite, stage, or reformat unrelated user changes.
- Do not fix unrelated lint warnings, refresh unrelated baselines, or perform broad refactors.
- Do not create destructive migrations, reset databases, rewrite migration history, or reseed shared environments.
- Do not invent customer metrics, risk scores, credit decisions, invoice identity, accounting statuses, reconciliation state, delivery outcomes, consent, or compliance claims when authoritative data is unavailable.
- Do not treat mutable `SalesOrder` data as immutable invoice-grade truth when `CustomerReceivableDocument` is the authoritative posted receivable source.

- Do not treat capped recent arrays or `Customer.currentBalance` alone as a complete receivable statement or certified subledger roll-forward.
- Do not silently aggregate different currencies. Group totals, aging, exposure, statements, exports, and comparisons by currency unless an authoritative, dated FX policy and conversion service already exist.
- Use exact monetary representations at service boundaries. Preserve `Prisma.Decimal` or canonical decimal strings and currency precision; do not introduce floating-point accounting calculations.
- Never expose full tax identifiers, private notes, contact details, payment destinations, provider references, statement tokens, document hashes, or internal evidence identifiers by default.
- Require explicit consent and the existing secure provider envelope for customer communications. A stored email address or phone number is not proof of delivery consent.
- Preserve immutable documents, lifecycle evidence, allocations, compensating reversals, statements, access logs, and recipient actions. Never edit or delete financial proof in place.
- Treat the 2026-08-09 customer referral loop as pilot-ready evidence, not blanket production-release certification. Re-verify current repository and target-environment truth.

Repository evidence to inspect first:

1. Canonical customer routes:

- `app/[locale]/(dashboard)/dashboard/customers/page.tsx`
- `app/[locale]/(dashboard)/dashboard/customers/new/page.tsx`
- `app/[locale]/(dashboard)/dashboard/customers/[id]/page.tsx`
- `app/[locale]/(dashboard)/dashboard/customers/[id]/edit/page.tsx`
- `app/[locale]/(dashboard)/dashboard/customers/[id]/orders/page.tsx`
- `app/[locale]/(dashboard)/dashboard/customers/[id]/orders/CustomerOrdersClientPage.tsx`
- `app/[locale]/(dashboard)/dashboard/customers/[id]/statement/page.tsx`
- `app/[locale]/(dashboard)/dashboard/customers/layout.tsx`
- `app/[locale]/(dashboard)/dashboard/customers/loading.tsx`
- `app/[locale]/(dashboard)/dashboard/customers/error.tsx`

2. Customer UI, client state, and data access:

- `components/customers/CustomerManagementDashboard.tsx`
- `components/customers/CustomerQuickActions.tsx`
- `components/customers/CustomerStatementWorkflow.tsx`
- `hooks/useCustomerManagement.ts`
- `hooks/useCustomerQueries.ts`
- `types/customerTypes.ts`
- `validations/customer.ts`

3. Customer actions, schemas, and service-owned read models:

- `actions/customers/customer-management-actions.ts`
- `actions/customers/customerActions.ts`
- `actions/customers/customerAction2.ts`
- `services/customer/customer.service.ts`
- `services/customer/customer.schemas.ts`

- `services/customer/__tests__/customer-legacy.service.test.ts`
- Identify overlapping legacy and management action paths and prevent divergent authorization, validation, lifecycle, and error semantics.

4. Sales, POS, receipts, returns, and payments:

- `app/[locale]/(dashboard)/dashboard/sales/`
- `app/[locale]/(dashboard)/dashboard/pos/`
- `services/pos/pos.service.ts`
- `services/accounting/postings/post-sale.ts`
- `services/accounting/postings/post-payment.ts`
- `services/accounting/postings/post-refund.ts`
- Inspect the canonical sales-order, receipt, return/refund, and payment routes and services discovered from source rather than assuming filenames.

5. Accounts-receivable overview and open-item history:

- `app/[locale]/(dashboard)/dashboard/finance/receivables/page.tsx`
- `app/[locale]/(dashboard)/dashboard/finance/receivables/history/page.tsx`
- `components/finance/FinanceSpecializedLedgerSurfaces.tsx`
- `components/finance/AROpenItemsHistoryWorkbench.tsx`
- `hooks/useAROpenItemsHistoryWorkbench.ts`
- `actions/finance/ar-history.actions.ts`
- `services/accounting/ar-open-item.service.ts`
- `services/accounting/customer-ledger.service.ts`

6. Receivable-document and settlement truth:

- `services/accounting/customer-receivable-document.service.ts`
- `services/accounting/customer-receivable-lifecycle.service.ts`
- `services/accounting/customer-receivable-backfill.service.ts`
- `services/accounting/customer-settlement.service.ts`
- `services/accounting/customer-settlement.schemas.ts`
- `services/accounting/customer-settlement-reversal.service.ts`
- `actions/finance/customer-settlement.actions.ts`
- Focused tests under `services/accounting/__tests__/` and `actions/finance/__tests__/`.

7. Customer statement, secure access, recipient actions, and delivery:

- `actions/accounting/customer-statement.actions.ts`
- `services/accounting/customer-statement.service.ts`
- `services/accounting/customer-statement.schemas.ts`
- `services/accounting/customer-statement-token.ts`
- `services/accounting/customer-statement-access.service.ts`
- `services/accounting/customer-statement-recipient-action.service.ts`
- `services/accounting/customer-statement-delivery.service.ts`
- `services/accounting/customer-statement-delivery-envelope.ts`

- `services/communication/customer-statement-delivery.provider.ts`
- `services/communication/customer-statement-delivery-worker.service.ts`
- `app/customer-statement/[statementId]/page.tsx`
- `app/customer-statement/[statementId]/CustomerStatementPortal.tsx`
- `app/api/customer-statements/[statementId]/route.ts`
- `app/api/customer-statements/[statementId]/actions/route.ts`

8. Persistence and migration evidence:

- `prisma/schema.prisma`, especially `Customer`, `SalesOrder`, `Payment`, `PaymentRefund`, `CustomerLedgerEntry`, `CustomerReceivableDocument`, `receivable lifecycle state`, `customer settlements/allocations`, `statement snapshots/tokens/access logs`, `recipient actions`, and `delivery records`.
- `Customer/receivable/statement migrations` under `prisma/migrations/`, including the 2026-08-08 and 2026-08-09 customer settlement, receivable-document, statement, access, and delivery foundations.
- `prisma/migration-history-checksum-approvals.json` and current migration safety reports if schema work is proposed.

9. Current review and release evidence:

- `what-next/referrals/CUSTOMER_LEDGER_BALANCE_INTEGRITY_KERNEL_SLICE_434_REPORT_2026-08-08.md`
- `what-next/referrals/CUSTOMER_SETTLEMENT_ALLOCATION_SOURCE_FOUNDATION_SLICE_435_REPORT_2026-08-08.md`
- `what-next/referrals/CUSTOMER_SETTLEMENT_COMPENSATING_REVERSAL_SLICE_436_REPORT_2026-08-08.md`
- `what-next/referrals/CUSTOMER_SETTLEMENT_REVERSAL_PROTECTED_ACTION_SLICE_437_REPORT_2026-08-08.md`
- `what-next/referrals/CUSTOMER_REFERRAL_TAKEOFF_READINESS_2026-08-09.md`
- `what-next/referrals/CUSTOMER_STATEMENT_ENTITLEMENT_ACTIVATION_2026-08-09.md`
- `what-next/referrals/CUSTOMER_REFERRAL_MIGRATION_BASELINE_REPAIR_2026-08-09.md`
- `what-next/referrals/customer-referral-pilot-evidence-readiness.md`
- `what-next/transaction-history/runs/th-ar-accounting-prerequisites-20260717-028/slices/ap-ar/01-architecture-gate.md`
- `what-next/transaction-history/runs/th-ar-accounting-prerequisites-20260717-028/slices/ap-ar/02-security-proof-gate.md`
- `what-next/transaction-history/runs/th-ar-accounting-prerequisites-20260717-028/slices/ap-ar/03-accounting-control-gate.md`
- `what-next/transaction-history/runs/th-ar-open-item-history-implementation-20260717-029/slices/ap-ar/06-frontend-delivery.md`
- `docs/architecture/system/ACTIVE_SURFACE_MAP.md`
- `docs/modules/AQSTOQFLOW_CANONICAL_MODULE_VOCABULARY_2026-07-12.md`
- `docs/domains/sales/SALES_SYSTEM_MODERNIZATION_ANALYSIS.md`
- Treat older blocked reports as historical evidence. Reconcile them with later implementation and certification reports instead of repeating stale conclusions.

10. RBAC, entitlement, export, and public-boundary evidence:

- `config/permissions.ts`
- `lib/security/rbac-permissions.ts`
- `services/modules/module-entitlement.service.ts`
- `services/security/export-safety.service.ts`
- `services/history/transaction-history-cursor.ts`

- Existing public identity, rate-limit, abuse, token, consent, secrets, audit, and release-gate helpers used by the statement workflow.

11. Architecture evidence:

- graphify-out/POST_SLICE_438_GRAPH_REFRESH_2026-08-09.md
- graphify-out/GRAPH_REPORT_app.md
- graphify-out/GRAPH_REPORT_components.md
- graphify-out/GRAPH_REPORT_actions.md
- graphify-out/GRAPH_REPORT_hooks.md
- graphify-out/GRAPH_REPORT_types.md
- Inspect action graph Community 3, which contains legacy customer actions and order access.
- Inspect action graph Community 13, which contains customer management and analytics actions.
- Inspect component graph Community 0, which includes customer form and export helpers, but verify inferred relationships against source because its cohesion is low.
- Treat source code, schema, migrations, focused tests, and authenticated runtime behavior as authoritative when graphs or reports are incomplete.

Known baseline to verify, not blindly assume:

- /dashboard/customers is the current canonical customer-management route family.
- The customer management service already exposes directory and detail-analytics read models, but the management list is capped and filtered client-side, and detail analytics currently uses recent capped arrays. Verify scale, completeness, pagination, and truth-label implications.
- CustomerManagementDashboard.tsx currently exports customer email, phone, credit exposure, balance, and sales data directly in the browser without a dedicated server export authorization and audit boundary. Verify and remediate default minimization, privileged scope, filter parity, fresh authentication, and audit evidence.
- CustomerQuickActions.tsx simulates customer export with a timer, and the customer orders page contains a visible export control marked for future logic. These are known non-functional candidates; confirm current source and remove, hide, or implement them through a real authorized export boundary.
- Customer create, list, detail, edit, and order routes have page-level permission tests, but customer management actions currently accept an organization identifier from the client and perform their own permission checks. Verify service/action consistency, module entitlement, no-enumeration behavior, and direct-action resistance.
- Canonical vocabulary assigns customer management to the sales module. Statement creation and sharing are separately protected by customers.read, accounting.exports.create, and the accounting module. Finance receivables use finance permissions. Preserve these boundaries.
- Customer removal already distinguishes archived from deactivated based on sales-order or ledger history. Verify the policy against receivable documents, settlements, statements, access logs, delivery records, refunds, and other retained evidence, not only the current two counts.
- Posted receivable documents, lifecycle evidence, settlements, allocations, compensating reversal, immutable statements, signed access, public redaction, recipient actions, and consented delivery exist in the 2026-08-09 baseline. Preserve their evidence and idempotency contracts.
- The AR open-item service supports customer filtering, effective asOf, recordedThrough, currency, due dates, aging, allocations, evidence grade, document/state hashes, and conservation checks. Verify that summaries never combine currencies and that UI/export semantics retain the same temporal and evidence context.
- The customer statement application entitlement boundary was certified, but production migration adoption, secrets, provider credentials, target-organization provisioning, remote target evidence, and real-user pilot evidence remained external blockers on 2026-08-09.
- Current repository state may have changed since these reports. Inspect before implementing and document superseded evidence.

Execution tasks:

1. Establish current truth

- Run `git status --short` and record pre-existing changes relevant to the customer slice.
- Trace every customer route through components, hooks, actions, services, Prisma models, module checks, provider boundaries, workers, and tests.
- Produce a workflow matrix with implemented, partial, broken, missing, duplicated, unsafe, blocked by external configuration, or intentionally deferred for each visible and server-side capability.
- Verify behavior through source, schema, tests, authenticated routes, public statement routes, and focused provider/worker tests—not filenames or old reports alone.
- Identify canonical customer, sales-order, receivable, settlement, and statement surfaces and the explicit handoff contract between them.
- Inventory `customerActions.ts`, `customerAction2.ts`, and `customer-management-actions.ts`. Preserve compatibility only where required; prevent divergent validation, authorization, lifecycle, and response semantics.
- Record which historical findings are now remediated, which remain valid, and which require target-environment evidence.

2. Confirm domain, module, and route ownership

- Keep customer directory/profile lifecycle under the canonical `sales` owner.
- Keep finance receivable overview, open items, aging, collection, settlement, and reversal under finance/AR permissions and entitlement.
- Keep immutable statement creation, export/share, revocation, and proof under Accounting permissions and `moduleSlug: "accounting"`.
- Do not grant statement, settlement, proof, or export authority merely because a user can read a customer profile.
- Ensure direct navigation and refresh work for list, new, detail, edit, orders, statement, receivables overview/history, and permitted public statement routes.
- Preserve locale-aware URLs, deep links, query filters, back navigation, and browser history.

3. Complete the customer directory

- Display tenant-scoped customer records using the service-owned management read model.
- Ensure search, sorting, pagination or bounded-result disclosure, active/inactive filter, activity filter, locale filter, credit-exposure filter, and clear-filter behavior work against authoritative data.
- If the current 500-row cap remains, make the cap explicit and prevent the UI, summaries, and exports from implying completeness. Prefer server pagination/filter parity when the workflow requires full-directory operation.
- Keep create, view analytics, edit, orders, statements, archive/deactivate, export, copy, contact, and navigation actions permission- and entitlement-aware.
- Show useful summary cards derived from authoritative server data, including active customers, open sales orders, unpaid/open receivables, overdue exposure, credit-limit exceptions, and customer count where supported.
- Separate operational sales totals from posted receivable truth. Label period, source, currency, completeness, `asOf`, and `recordedThrough` where material.
- Provide loading, initial-empty, filtered-empty, error, permission-denied, module-locked, no-active-organization, capped-result, stale-data, partial-evidence, and retry states.

4. Verify and complete customer creation

- Direct navigation to `/dashboard/customers/new` must work after refresh.
- Enforce `customers.create` and the `sales` module boundary before exposing or mutating customer data.
- Validate all existing customer fields through shared server schemas.

- Cover name, organization-scoped code, email, phone, address, tax identifier, payment terms, credit limit, preferred locale, notes, and active status when supported by the current model and policy.
- Normalize fields consistently across every customer action path without erasing meaningful values.
- Prevent accidental duplicate submission and support idempotent behavior where established command patterns require it.
- Display field-level and form-level validation messages without leaking cross-tenant duplicate or existence data.
- Persist through the canonical action-to-service boundary using the server-derived organization.
- After success, invalidate relevant directory, customer-picker, sales/POS, receivable, and analytics queries and navigate to the created customer's detail page.
- Confirm the new customer appears in permitted POS and sales-order selectors without exposing customers from another organization.

5. Verify and complete customer editing

- Direct navigation to `/dashboard/customers/[id]/edit` must load the correct customer after refresh.
- Enforce `customers.update` and the `sales` module boundary.
- Prepopulate the form from the authoritative customer record.
- Handle nonexistent, inactive, archived, cross-tenant, and permission-denied customers safely without enumeration.
- Preserve validation parity between create and edit.
- Persist through the canonical update service and refresh directory, detail, order, POS selector, AR, and statement-adjacent caches after success.
- Provide saving, success, failure, retry, cancel, and unsaved-change behavior consistent with established Stoquify patterns.
- Treat changes to name, tax ID, address, locale, terms, and credit policy prospectively. Never mutate immutable receivable-document or statement snapshots that intentionally preserve historical customer identity.
- Do not introduce a new concurrency/versioning mechanism unless a proven stale-write defect requires it.

6. Complete customer lifecycle behavior

- Verify detail, edit, deactivate, reactivate, archive, and any existing restore behavior.
- Preserve the core rule that customers with material sales, receivable, settlement, payment, refund, ledger, statement, access, delivery, recipient-action, referral, or audit history are deactivated rather than deleted or hidden destructively.
- Allow archival only when the established retention policy proves the customer is unused and has no dependent evidence.
- Expand or document the lifecycle-history check if the existing decision considers only sales orders and ledger entries.
- Require a clear confirmation dialog that accurately explains whether the operation will archive or deactivate and what remains accessible.
- Keep historical orders, receipts, receivables, settlements, payments, statements, access logs, and audit evidence readable after deactivation to authorized users.
- Prevent inactive/deactivated customers from new credit transactions when policy requires it, while preserving read access and approved settlement of existing obligations.
- Add explicit tests for archive, deactivate, reactivation, history retention, and cross-tenant denial branches.
- Record auditable lifecycle events through the existing audit/business-event facilities when those facilities own the boundary.

7. Complete a coherent customer detail workspace

- Build or preserve a coherent, role-aware customer workspace rather than unrelated cards or modal-only analytics.
- Use accessible tabs, subroutes, or the established system navigation pattern. Keep high-level operational data separate from accounting proof.

A. Overview

- Customer identity, active state, code, contact summary, preferred language, payment terms, credit limit, and safe notes summary.
- Current operational balance, posted open receivables, overdue amount, available credit, open orders, unpaid orders, last order, last payment/settlement, and statement availability where authoritative.
- Clearly identify source, currency, period, completeness, evidence grade, freshness, `asOf`, and `recordedThrough` for every financial metric.
- Flag over-limit exposure, overdue balances, inactive customers with open obligations, missing posted-document evidence, stale projections, and other server-derived exceptions.
- Do not derive or label creditworthiness, collection probability, loyalty, churn, or lifetime value without approved definitions and authoritative data.

B. Sales orders, POS, receipts, returns, and refunds

- Show customer-filtered orders with order number, channel/location, order date, due date, status, payment status, amount, currency context, fulfillment/receipt state, and navigable links where supported.
- Connect to canonical sales-order, receipt, POS, return, and refund detail surfaces.
- Distinguish mutable operational orders from immutable posted receivable documents.
- Replace the known simulated order export with a real authorized export or remove/hide it until the server boundary exists.
- Preserve server pagination/filtering for complete history; do not use a client-only subset to imply a full account history.

C. Receivables and aging

- Reuse `services/accounting/ar-open-item.service.ts` and the AR history action/workbench rather than duplicating aging or open-balance calculations inside customer components.
- Filter by `customerId` at the server boundary and preserve signed filter/cursor semantics where implemented.
- Show immutable document number/version, invoice date, due date, currency, opening amount, paid/allocated amount, open amount, status, days past due, aging bucket, evidence grade, and safe source links.
- Preserve effective `asOf` and `recordedThrough` semantics and expose them in the UI and export.
- Group every summary and aging bucket by currency. Never add XAF, EUR, USD, or other currencies together without an approved FX basis.
- Expose missing lifecycle evidence, non-conservation, incomplete backfill, stale projection, or unsupported source states as explicit exceptions instead of silently falling back.

D. Payments, settlements, allocations, and reversals

- Show customer-filtered settlement and payment history from authoritative services with reference, effective date, recorded date, method, status, amount, currency, allocated documents, unallocated amount, processor/reconciliation evidence state, reversal lineage, and safe source links.
- Redact bank, card, mobile-money, token, provider, and destination data according to existing policy.
- Preserve idempotency, compare-and-set state transitions, allocation conservation, accounting journal linkage, ledger entries, business events, and compensating reversal semantics.
- Do not mutate or delete posted settlements or allocations in place.
- Require fresh authentication, `finance.receivables.collect` or `.reverse`, segregation of duties, maker-checker, or other existing high-risk controls where policy requires them.
- Do not claim reconciled, settled, cleared, reversed, or certified status without the required provider, subledger, and accounting evidence.

E. Statements and customer-facing proof

- Link authorized users to `/dashboard/customers/[id]/statement` through the existing Accounting entitlement boundary.

- Generate immutable, content-hashed statements from the posted receivable/open-item source at explicit period and knowledge cutoffs.
- Show statement number, period, currency, opening, movements, closing, reconciliation status, hashes/evidence only at the appropriate privileged disclosure level, and explicit completeness state.
- Preserve signed, expiring, revocable external access; append-only access logging; redacted public rendering; and no-enumeration behavior.
- Preserve recipient dispute and promise-to-pay commands as append-only, idempotent, rate-limited actions that cannot mutate accounting truth.
- Require explicit delivery consent, sealed provider envelopes, channel-safe destinations, and auditable delivery states.
- Keep public routes independent from broad authenticated dashboard data. Never expose tenant IDs, raw customer IDs, secrets, provider payloads, internal notes, full contact data, document hashes, or unrelated account history.
- Make statement revocation, expiry, wrong-token, replay, rate-limit, provider-failure, consent-withdrawal, bounce/failure, and unavailable-channel states clear and recoverable without weakening security.

8. Complete customer credit and receivable controls

- Trace how credit limits and payment terms are enforced across customer creation/editing, POS, sales orders, receivable issuance, settlement, and reporting.
- Keep credit decisions server-side and transactionally safe. Do not rely on a dashboard warning to prevent an unauthorized credit sale.
- Define authoritative treatment for over-limit sales, overrides, approval evidence, inactive customers, overdue accounts, refunds, credit notes, write-offs, and bad debt before exposing corresponding controls.
- If any control is not implemented, report it as a gap or intentional deferral. Do not simulate it.
- Reconcile customer subledger/open-item totals to the appropriate AR control-account evidence when the accounting backbone supports the tie-out.
- Prove opening plus signed movements equals closing at the same effective and recorded-through boundaries.
- Preserve fiscal period/close behavior and prevent backdated actions from silently invalidating certified statements or close evidence.

9. Eliminate fake, unsafe, and inconsistent export/contact actions

- Inventory every customer, order, receivable, payment, and statement export or clipboard action.
- Remove or disable timer-based simulated exports and success notifications that occur before a real result exists.
- Route exports through existing server-owned export safety, RBAC, entitlement, fresh-authentication, tenant/filter binding, redaction, row-limit, and audit facilities.
- Default customer exports must omit or mask contact details, tax IDs, private notes, credit exposure, provider/payment data, public tokens, hashes, and proof identifiers unless an existing privileged policy explicitly allows selected fields.
- Sensitive exports require deliberate scope selection, clear purpose, appropriate permission, fresh authentication, and an auditable outcome.
- Preserve table/export filter parity and disclose truncation, currency grouping, asOf, recordedThrough, evidence grade, and data-source completeness.
- Record actor, organization, surface, filters, selected fields, row count, timestamp, outcome, and correlation through the established audit mechanism without logging raw sensitive content.
- Keep clipboard actions minimal and prevent accidental copying of hidden metadata.
- Treat `mailto:`/`tel:` or provider-based contact actions as real actions only when values exist, permission and consent policies allow them, and the UI does not claim that communication was delivered.

10. UI and design-system quality

- Match the established Stoquify shell, page header, cards, typography, spacing, colors, tables, filters, buttons, badges, dialogs, drawers, notifications, locked-module states, and error states.

- Reuse shared design-system components and finance/history primitives instead of adding one-off visual systems.
- Make dense customer, order, AR, settlement, and statement tables readable and responsive on desktop, tablet, and mobile.
- Use progressive disclosure for contact data, credit exposure, financial proof, hashes, allocations, and detailed history.
- Maintain visible focus, logical keyboard order, accessible names, form associations, table semantics, dialog/drawer focus management, live-region announcements, touch target size, and reduced-motion behavior.
- Do not encode status, aging, risk, consent, or exception using color alone.
- Complete English and French copy. Do not leave raw translation keys, unaccented placeholder French, mixed-language surfaces, or English-only public/customer-facing states.
- Preserve the customer's context, filters, scroll/selection, and return destination across create, edit, order, receivable, settlement, statement, and public-to-authenticated handoffs where safe.
- Use human-readable accounting language with an accessible evidence drill-down; do not mislabel a customer subledger as the general ledger.

11. Security, privacy, abuse resistance, and controls

- Enforce organization scope at every query and mutation; derive the authoritative organization from the authenticated server context.
- Preserve `customers.read`, `.create`, `.update`, `.delete`, `orders.read`, `analytics.read`, and `communication.send` only where they are currently canonical. Do not widen permissions or inherit write/export authority from read access.
- Preserve finance receivable permissions such as `finance.receivables.read`, `.collect`, `.reverse`, and finance report export permissions where current policy requires them.
- Preserve `accounting.exports.create` plus enforced `moduleSlug`: "accounting" for statement creation, delivery, and revocation.
- Enforce `moduleSlug`: "sales" for customer-management surfaces if the canonical catalog and current entitlement facilities support enforced access; document compatibility mode rather than silently bypassing entitlement.
- Add cross-tenant and no-enumeration denial tests for list, detail, update, analytics, orders, AR history, settlement, reversal, statement creation, statement delivery, revocation, exports, public access, recipient actions, and lifecycle operations.
- Bind public tokens to the intended statement, expiry, status, and revocation state. Store only hashes where existing design requires them and use constant-safe verification patterns.
- Rate-limit and abuse-protect public statement view, dispute, promise-to-pay, referral, invite, and delivery-triggering surfaces.
- Validate SSRF, open redirect, CSV/spreadsheet injection, HTML injection, log injection, replay, token leakage, cache leakage, and provider callback risks relevant to the implemented slice.
- Preserve CSRF/origin, secure-cookie, CSP, cache-control, referrer, and response-header protections used by public or authenticated routes.
- Do not expose raw database, provider, cryptographic, migration, or internal error details to users.
- Never log statement tokens, encryption keys, full contact destinations, financial payloads, tax IDs, or raw provider envelopes.

12. Reliability, observability, performance, and operations

- Ensure customer, AR, settlement, statement, delivery, and public-access operations emit structured, redacted, correlation-aware logs and auditable business outcomes.
- Preserve idempotent retries for settlements, reversals, statement generation, recipient actions, referral attribution, and delivery queue operations.
- Define retry, dead-letter, poison-message, provider-timeout, partial-failure, and operator-recovery behavior for delivery workers without duplicate delivery or duplicate financial effects.
- Monitor statement generation/delivery failures, access denials, token abuse, settlement/reversal failures, conservation errors, AR tie-out drift, queue lag, and provider health.

- Avoid N+1 queries, unbounded customer/history loads, repeated client aggregation, and full sensitive dataset materialization in the browser.
- Preserve stable pagination/cursor behavior and document explicit caps.
- Do not weaken fail-closed production secret, origin, migration, provider, entitlement, or pilot evidence gates to make local checks green.

13. Testing requirements

Add or update focused tests for:

- Customer list/new/detail/edit/orders/statement direct-load, authorization, module-entitlement, no-active-organization, and locked-module behavior.
- Successful customer creation, validation failure, duplicate handling, double submission, and cross-tenant non-enumeration.
- Edit prepopulation, successful persistence, immutable snapshot preservation, cache invalidation, and cross-tenant denial.
- Archive versus deactivate semantics across every retained history type; reactivation and inactive-customer restrictions.
- Customer-specific order/POS/receipt/return/refund filtering and navigation.
- AR open-item filtering, as-of/recorded-through semantics, aging boundaries, currency separation, conservation, allocation, reversal, cancellation/void, missing lifecycle evidence, and cursor/filter integrity.
- Customer summary and detail analytics completeness labels, caps, currency correctness, and no client-owned accounting truth.
- Settlement idempotency, partial/multi-item allocation, compare-and-set conflict, accounting/ledger linkage, compensating reversal, and replay resistance.
- Statement immutability, content hash, currency, opening/movement/closing reconciliation, idempotent generation, and source completeness.
- Statement entitlement, signed access, expiry, revocation, wrong token, redaction, cache safety, access logging, rate limiting, dispute, and promise-to-pay behavior.
- Explicit consent, encrypted delivery envelope, redacted destination, provider success/failure/retry, revocation, and worker idempotency.
- Default export minimization, privileged export controls, fresh authentication, filter parity, truncation disclosure, CSV injection defense, and auditable export behavior.
- Removal or real implementation of every known simulated export; notifications must reflect actual outcomes.
- Loading, empty, filtered-empty, error, retry, inactive, permission-denied, stale, capped, partial-evidence, and provider-unavailable states.
- English and French content, keyboard behavior, dialog/drawer focus, screen-reader announcements, responsive authenticated flows, and public statement accessibility.

14. Browser and target-boundary verification

Execute authenticated end-to-end verification for:

- Opening the customer directory and verifying real summary/filter behavior.
- Creating a uniquely identifiable test customer and confirming it appears in the directory and permitted customer pickers.
- Opening the customer's detail workspace and editing the record; verify persisted changes after refresh.
- Opening customer-filtered sales/POS order history and a real order/receipt destination.
- Opening customer-filtered AR history; verify due date, aging, currency, allocation, asOf, recordedThrough, evidence grade, and source links.
- Exercising an appropriate settlement/allocation and, only with safe fixtures and authorization, a compensating reversal.

- Generating an immutable customer statement, verifying its reconciliation, queueing consented delivery to a safe test recipient, opening the redacted signed route, submitting an allowed recipient action, and revoking access.
- Confirming wrong-token, expired, revoked, cross-tenant, restricted-role, unentitled-module, and no-enumeration behavior.
- Exercising archive/deactivate behavior with appropriate isolated fixtures while preserving history.
- Verifying every export/contact control has a real result or is absent/disabled with an honest explanation.
- Capturing desktop, tablet, and mobile screenshots for authenticated customer/AR surfaces and the public statement route.
- Checking browser console, network failures, cache headers, and accessible-name/focus behavior.

Store screenshot evidence under:

what-next/screenshots/customer-workflow-completion-2026-08-09/

Execution checklist:

1. Discovery

- Record dirty-worktree state and all relevant current evidence.
- Build route/action/service/schema/module/permission/provider ownership maps.
- Reconcile older AR prerequisite findings with the newer posted-document, settlement, and statement foundations.

2. Gap analysis

- Classify each workflow and visible control.
- Separate code gaps, data/migration gaps, target-configuration gaps, provider/secret gaps, and real-user evidence gaps.
- Rank findings by financial truth, tenant/privacy exposure, irreversible evidence risk, customer harm, and operational impact.

3. Implementation

- Fix the smallest coherent dependency-ordered slice.
- Prefer service/read-model/action contracts before UI changes.
- Preserve certified foundations and add migrations only when the current schema cannot safely represent required truth.

4. Verification

- Run focused unit/action/service/component/route tests first.
- Run wider static, policy, migration, provider, browser, and build gates in proportion to touched boundaries.
- Record every command as passed, failed, skipped, timed out, or blocked, with exact evidence.

5. Handoff

- Save the completion report and screenshots at the specified locations.
- Identify residual risks, deferred work, external approvals/configuration, rollback/recovery guidance, and the next smallest executable slice.

Verification commands:

Run focused checks first, then wider gates in proportion to the changes.

- ```
npm test -- --runInBand services/customer/__tests__/customer-legacy.service.test.ts
app/[locale]/(dashboard)/dashboard/customers/__tests__/pages.test.tsx
app/[locale]/(dashboard)/dashboard/customers/[id]/orders/__tests__/page.test.tsx
```

- `npm test -- --runInBand services/accounting/__tests__/customer-ledger.service.test.ts`  
`services/accounting/__tests__/ar-open-item.service.test.ts`  
`services/accounting/__tests__/customer-receivable-document.service.test.ts`  
`services/accounting/__tests__/customer-receivable-lifecycle.service.test.ts`
- `npm test -- --runInBand services/accounting/__tests__/customer-settlement.service.test.ts`  
`services/accounting/__tests__/customer-settlement-reversal.service.test.ts`  
`actions/finance/__tests__/customer-settlement.actions.test.ts`
- `npm test -- --runInBand services/accounting/__tests__/customer-statement.service.test.ts`  
`services/accounting/__tests__/customer-statement-token.test.ts`  
`services/accounting/__tests__/customer-statement-access.service.test.ts`  
`services/accounting/__tests__/customer-statement-recipient-action.service.test.ts`
- `npm test -- --runInBand services/accounting/__tests__/customer-statement-delivery.service.test.ts`  
`services/accounting/__tests__/customer-statement-delivery-envelope.test.ts`  
`services/communication/__tests__/customer-statement-delivery.provider.test.ts`  
`services/communication/__tests__/customer-statement-delivery-worker.service.test.ts`
- `npm test -- --runInBand actions/accounting/__tests__/customer-statement.actions.test.ts`  
`components/customers/__tests__/CustomerStatementWorkflow.test.tsx`  
`app/[locale]/(dashboard)/dashboard/customers/[id]/statement/__tests__/page.test.tsx`
- `npm test -- --runInBand app/customer-statement/[statementId]/__tests__/CustomerStatementPortal.test.tsx`  
`app/api/customer-statements/[statementId]/__tests__/route.test.ts`  
`app/api/customer-statements/[statementId]/actions/__tests__/route.test.ts`
- Run all new customer-management action, export, component, route, hook, AR workbench, accessibility, and browser tests added by this work.
- `npm run typecheck`
- `npm run lint`
- `npm run report:trust:export:gate`
- `npm run policy:gates`
- `npm run build:app`
- Run `npm run prisma:validate` and the focused migration safety/history gates if Prisma schema or migration files are touched.
- Run `npm run referral:postgres:smoke` only against an explicitly configured isolated PostgreSQL database when receivable, settlement, statement, access, recipient, referral, or delivery persistence changes.
- Run `npm run referral:pilot:evidence:report` for pilot/release-readiness work. Do not fabricate the external production inputs required for the fail-closed gate.

Do not run `npm run reset`, destructive reseeding, destructive migration commands, or production release commands without the required target configuration and approvals.

## Expected artifacts:

- Surgical implementation changes across the existing customer, sales/POS handoff, AR, settlement, payment, and statement workflow.
- Focused unit, action, service, component, route, API, worker, security, accessibility, and browser tests.
- Desktop, tablet, mobile, and public-statement screenshot evidence.
- A completion report saved as:

`what-next/STOQUIFY_CUSTOMER_WORKFLOW_COMPLETION_REPORT_2026-08-09.md`

## The report must contain:

- Executive decision: complete, conditionally complete, partially complete, blocked, or intentionally deferred, with evidence.
- Current-state and final-state workflow matrix.

- Files changed and why, separated from pre-existing dirty-worktree changes.
- Canonical route/action/service/module ownership decisions and treatment of overlapping legacy customer actions.
- Data-source map for every displayed metric, order, open item, payment, settlement, statement, and export field.
- Currency, monetary precision, period, asOf, recordedThrough, evidence grade, completeness, and reconciliation semantics.
- RBAC, module-entitlement, tenant-isolation, fresh-authentication, no-enumeration, redaction, consent, abuse-resistance, and audit evidence.
- Archive/deactivate/retention decision evidence.
- Export inventory and proof that simulated or unsafe exports were removed or replaced.
- Commands executed with passed, failed, skipped, timed-out, or blocked status.
- Browser scenarios, screenshot locations, console/network results, and accessibility evidence.
- Migration, provider, secret, origin, environment, and real-user pilot dependencies.
- Remaining limitations, residual risks, external approvals, explicitly deferred work, and rollback/recovery notes.
- No unsupported enterprise, security, accounting, privacy, accessibility, legal, compliance, production-readiness, or release-certification claims.

## Risk controls:

- Tenant isolation: every customer, order, receivable, settlement, statement, access, delivery, and export read/write remains organization-scoped.
- RBAC and entitlement: customer, finance, Accounting, export, communication, and public boundaries remain explicit and independently enforced.
- Accounting truth: posted documents, lifecycle evidence, ledger movements, allocations, settlements, reversals, statements, and close/tie-out proof remain immutable and reconcilable.
- Monetary integrity: exact decimals, currency precision, per-currency aggregation, and conservation equations are preserved.
- Privacy and communications: contact/tax/payment data is minimized; consent, redaction, secure delivery, retention, and audit policies are enforced.
- Public abuse: signed-token expiry/revocation, no enumeration, rate limits, safe caching, recipient-action idempotency, and audit logs remain intact.
- Migration safety: no destructive reset or silent history rewrite; fresh and populated-database paths remain distinct and approved.
- Provider reliability: sealed envelopes, retry/idempotency, failure handling, and dead-letter/operator recovery prevent duplicate or false delivery claims.
- UI regression: shared dashboard semantics, EN/FR behavior, responsive layouts, focus, and complete states are preserved.
- Dirty worktree: unrelated user changes, baselines, warnings, and modules are not touched.
- Release evidence: local success is not promoted to production certification without target migrations, secrets, providers, entitlements, remote gates, and real-user evidence.

## Success criteria:

- Customer create works through the canonical server path and persists organization-scoped data.
- Customer edit loads and updates the correct record, remains correct after refresh, and does not rewrite historical snapshots.
- Directory, new, detail, edit, orders, statement, and permitted finance/AR routes work when opened directly.
- Customer management is protected by canonical customer permissions and the sales entitlement boundary; finance and Accounting capabilities remain separately gated.
- Directory search, sorting, filters, pagination/caps, empty/error states, and summaries are functional and honest about completeness.



- Customer analytics are derived from authoritative service-owned read models and label operational versus posted accounting truth.
- Sales/POS history is customer-filtered and links to real order, receipt, return, refund, or payment destinations where supported.
- AR history reuses the existing posted open-item service with customer filtering, signed filter/cursor semantics, asOf, recordedThrough, evidence grade, allocation history, and source links.
- All financial totals, aging, exposure, statements, and exports retain currency and precision context and never silently aggregate currencies.
- Settlement and reversal behavior preserves idempotency, allocations, ledger/accounting links, immutable evidence, and compensating correction semantics.
- Customer statements are immutable, content-hashed, reconcilable, permission- and entitlement-protected, and never overstate source completeness.
- Signed statement access is expiring, revocable, redacted, no-enumeration, rate-limited, safely cached, and auditable.
- Recipient dispute/promise-to-pay and delivery workflows preserve explicit consent, safe provider boundaries, idempotency, and honest outcome states.
- Archive/deactivate behavior preserves all financial, customer-facing, referral, delivery, and audit history.
- Every visible action is functional, permission-aware, entitlement-aware, state-complete, or intentionally unavailable with an honest explanation.
- No timer-based fake export, fake success notification, decorative filter, or UI-only financial workflow remains.
- Default exports minimize sensitive customer data; privileged exports require appropriate authorization/fresh authentication and are auditable.
- The workflow matches Stoquify's visual system across desktop, tablet, and mobile, with complete English and French experiences.
- Focused tests, typecheck, lint, export/report-trust gate, policy gates, and application build pass, or blockers are documented with exact evidence.
- Schema/migration, referral PostgreSQL, secret/provider, pilot, and release gates are run only when applicable and never bypassed.
- No unrelated files, warnings, modules, baselines, or user changes are modified.

## Non-goals:

- Do not rebuild sales, POS, accounting, payment reconciliation, customer referral, communications infrastructure, or the global design system broadly.
- Do not turn Stoquify customer management into a speculative general-purpose CRM, marketing-automation suite, loyalty platform, or AI sales assistant.
- Do not replace working services or certified trust foundations with component-local logic.
- Do not create new financial, credit-risk, collection, loyalty, lifetime-value, churn, or customer-success metrics without authoritative inputs and approved definitions.
- Do not merge operational sales orders with immutable receivable documents or statement snapshots.
- Do not add automated credit approvals, write-offs, bad-debt decisions, collection messaging, or legal notices without explicit policy, authority, human approval, and expert-reviewed provenance.
- Do not expose raw contact, tax, payment, provider, token, hash, referral, or audit data merely because it exists in the schema.
- Do not activate production providers, provision production secrets, contact real customers, or send real statements without explicit authorization and target controls.
- Do not rewrite migration history, adopt baseline migrations, or deploy pending migrations to a populated environment without DBA/release evidence and approval.

- Do not delete legacy action paths, routes, or stored records until compatibility, navigation, retention, and integration impact are proven.
- Do not perform unrelated cleanup, refactoring, translation rewrites, baseline refreshes, or lint remediation.

### **Optional next prompts after the completion report exists:**

- Run a narrow customer-export security ratification using the saved export inventory, cross-tenant tests, redaction evidence, fresh-authentication proof, and audit records.
- Run a customer/AR accounting tie-out certification for one isolated period and currency using posted receivable documents, settlements, reversals, customer subledger movements, and the AR control account.
- Run a customer-statement production-readiness gate against a named non-production target with approved migrations, secrets, HTTPS origin, provider credentials, Accounting entitlement, and remote browser evidence.
- Run an accessibility and bilingual content ratification of the completed authenticated customer/AR workflow and the public statement route using keyboard, screen-reader, responsive, and EN/FR evidence.